ΜΕΡΟΣ Α

Στο μέρος Α επιλέξαμε να υλοποιήσουμε τον αλγόριθμο ταξινόμησης quicksort.

Αρχικά, δημιουργήσαμε έναν πίνακα στον οποίο θα αποθηκεύουμε αντικείμενα τύπου City.

Κατασκευάσαμε μια μέθοδο compareTo, η οποία συγκρίνει κάθε δύο πόλεις μεταξύ τους, ώστε ύστερα να ταξινομήσουμε όλες τις πόλεις.

Για να υλοποιήσουμε την quicksort, δημιουργούμε τις παρακάτω 2 μεθόδους:

- ➔ Partition:

Αρχικά, ορίζουμε ως pivot την τελευταία θέση του πίνακα.

Δημιουργούμε δύο μεταβλητές-δείκτες, τις i και j, τις οποίες αρχικοποιούμε στις θέσεις (start -1) και start αντίστοιχα, όπου start είναι η πρώτη θέση του πίνακα.

Σε κάθε βήμα του αλγορίθμου αυξάνουμε την j κατά 1, και ο αλγόριθμος τρέχει μέχρι η j να πάρει ως τιμή τον δείκτη της προτελευταίας θέσης.

Σε κάθε επανάληψη του αλγορίθμου, ελέγχουμε μέσω της μεθόδου compareTo εάν η πόλη που βρίσκεται στο κελί με δείκτη j προηγείται της πόλης που βρίσκεται στο κελί με δείκτη pivot (εάν ισχύει αυτό, τότε **`cities[j].compareTo(pivot) = -1 < 0) .`**

Εάν όντως προηγείται η πόλη του j κελιού, αυξάνουμε το i κατά 1, και κάνουμε swap στις τιμές των κελιών i και j μέσω της βοηθητικής μεταβλητής temp. Στην temp αποθηκεύεται αρχικά η τιμή του κελιού i, μετά στο κελί i αποθηκεύεται η τιμή του κελιού j, και τέλος η τιμή της temp αποθηκεύεται στο κελί j. Στη συνέχεια, αυξάνουμε το j κάτα ένα για να προχωρήσουμε στην επόμενη επανάληψη.

Στην περίπτωση που η πόλη του j κελιού είναι μεγαλύτερη ή ίση, αγνοούμε την πόλη στο συγκεκριμένο κελί του πίνακα και αυξάνουμε το j κάτα ένα για να προχωρήσουμε στην επόμενη επανάληψη.

Η διαδικασία αυτή συνεχίζεται μέχρι το j να φτάσει τον δείκτη end του pivot, πράγμα που σημαίνει πως μόλις έχει ολοκληρωθεί ο έλεγχος στο προτελευταίο κελί, και κάνουμε swap την τιμή της θέσης pivot με την τιμή στην θέση i + 1 του πίνακα, όπου είναι και η τελική θέση του pivot μας, πάλι χρησιμοποιώντας την μεταβλητή temp. Η μέθοδος επιστρέφει την (νέα) θέση i της τιμής που προηγουμένως βρισκόταν στην θέση pivot.

- ➔ Quicksort:

Η δεύτερη μέθοδος παίρνει ως ορίσματα έναν πίνακα αντικειμένων (πόλεων) τύπου City (εδώ είναι ο πίνακας που περιέχει όλες τις πόλεις), καθώς και την αρχή και το τέλος ενός πίνακα. Ορίζουμε μια νέα ακέραια μεταβλητή pivot, που είναι η τιμή i που επιστρέφεται από την μέθοδο partition. Ύστερα, πραγματοποιούμε αναδρομικό quicksort στον δεξιό και αριστερό υποπίνακα του πίνακα που προκύπτει από το partitioning (αριστερός υποπίνακας = τα στοιχεία αριστερά του pivot, δεξιός υποπίνακας = τα στοιχεία δεξιά του pivot).

Η μέθοδος σταματάει να καλείται αναδρομικά όταν ο δείκτης end γίνει μικρότερος ή ίσος του δείκτη start, δηλαδή όταν ο πίνακας ή υποπίνακας τον οποίο πάμε να ταξινομήσουμε έχει 0 ή 1 στοιχεία (συνεπώς είναι ήδη ταξινομημένος).

ΜΕΡΟΣ Β

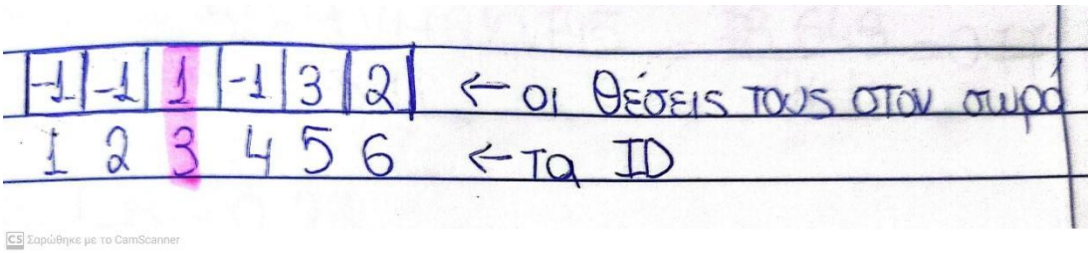
Σκοπός της μεθόδου remove είναι να αφαιρεί πόλεις με συγκεκριμένους κωδικούς ID από τον σωρό, χωρίς να παραβιαστεί η ελαχιστροστρεφής ιδιότητα του σωρού.

Αρχικά κατασκευάζουμε τον πίνακα positions, τον οποίο θα χρησιμοποιούμε για να βρούμε τον δείκτη της πόλης της οποία θέλουμε να αφαιρέσουμε. (Η πόλη αυτή θα έχει δοσμένο ID).

Εάν η πόλη δεν βρίσκεται στον σωρό, θα πάρει την τιμή -1 στον πίνακα. Δηλαδή όλες οι τιμές του πίνακα είναι αρχικά -1, αφού ο πίνακας είναι άδειος.

Ξεκινάμε την remove βρίσκοντας τον δείκτη (=position) της πόλης με δοσμένο ID που θέλουμε να αφαιρέσουμε. Αυτό γίνεται με την χρήση του πίνακα positions. Ο δείκτης δείχνει την θέση της πόλης στον σωρό.

Για παράδειγμα, η σχέση positions[3] = 1, όπου ID = 3 μας λέει ότι η πόλη με ID = 3 βρίσκεται στην πρώτη θέση του σωρού.



Σαρώθηκε με το CamScanner

Ύστερα, εντοπίζουμε την πόλη στην θέση του σωρού που αντιστοιχεί στον προηγούμενο δείκτη. Αυτήν την πόλη θέλουμε να αφαιρέσουμε από τον σωρό.

Κάνουμε swap της πόλης που θέλουμε να αφαιρέσουμε με την τελευταία πόλη του σωρού (δηλαδή την μεγαλύτερη, η οποία βρίσκεται στην θέση size του πίνακα position).

Η μέθοδος swap εκτελείται με την βοήθεια της βοηθητικής μεταβλητής tmp, η οποία λειτουργεί ακριβώς όπως η temp στο Μέρος Α.

Μετά την εκτέλεση του swap, μειώνεται το μέγεθος του σωρού κατά 1.

Ορίζουμε την θέση (position) της πόλης που βγάλαμε ως -1 στον πίνακα positions, για να δείξουμε ότι έχει αφαιρεθεί από τον σωρό.

Κάνουμε update της θέσης του στοιχείου που άλλαξε θέση με το πλέον αφαιρεμένο στοιχείο. Το update της θέσης (position) θα γίνει στον πίνακα positions.

Εάν χρειάζεται, κάνουμε swim και sink στο στοιχείο που άλλαξε θέση, ώστε να αποκατασταθεί η ιδιότητα του ελαχιστοστρεφούς σωρού.

Τέλος, η remove μας επιστρέφει την πόλη που αφαιρέσαμε.

Η πολυπλοκότητα της remove είναι O(logN), καθώς δεν χρειάζεται να αναζητήσουμε το στοιχείο (σε αυτήν την περίπτωση η πολυπλοκότητα θα ήταν O(N)). Ξέρουμε που βρίσκεται μέσω του δείκτη.

ΜΕΡΟΣ Γ

Το πρόγραμμα DynamicInfluenza_k_withPQ θέλουμε να διαβάζει γραμμές του αρχείου μας, και μετά την ανάγνωση κάθε γραμμής, να διατηρεί τις k πόλεις με τη μικρότερη πυκνότητα κρουσμάτων μέχρι εκείνη τη στιγμή σε μια ουρά προτεραιότητας.

Τα πρώτα k στοιχεία (πόλεις) που θα εισαχθούν στην ουρά θα εισαχθούν χωρίς κάποιον έλεγχο, αφού θα είναι εκ των πραγμάτων τα k μοναδικά (άρα αυτομάτως και καλύτερα) στοιχεία.

Όταν όμως θα έχουμε ήδη k ενεργά στοιχεία στην ουρά και θα πάμε να εισάγουμε ένα καινούριο, θα πρέπει να συγκρίνουμε τα στοιχεία μεταξύ τους, ώστε να αποφασίσουμε ποια k από τα (k+1) στοιχεία θα κρατήσουμε στην ουρά.

Εάν ορίσουμε με κάποιο τρόπο ότι τα στοιχεία της ουράς θα βρίσκονται πάντα σε φθίνουσα σειρά, θα μπορούμε να συγκρίνουμε το νέο στοιχείο με το πρώτο στοιχείο της ουράς (δηλαδή το μεγαλύτερο), και να εισάγουμε στην ουρά το νέο στοιχείο εάν είναι μικρότερο του πρώτου στοιχείου.

Για την υλοποίηση της παραπάνω ιδέας, μπορούμε να χρησιμοποιήσουμε την μέθοδο compareTo του ερωτήματος Α για την κατασκευή της ουράς. Η χρήση της μέθοδου ταξινομεί τα στοιχεία σε αύξουσα σειρά, επομένως θα πετύχουμε τα αντίθετα αποτελέσματα πολ/ζοντας την c1.compareTo(c2) με (-1).

Αν τελικά το νέο στοιχείο είναι μικρότερο του πρώτου στοιχείου της ουράς, χρειάζεται δηλαδή να εισαχθεί στην ουρά, θα αφαιρέσουμε το πρώτο στοιχείο της ουράς χρησιμοποιώντας την μέθοδο getMin, και ύστερα θα προσθέσουμε το νέο στοιχείο στην ουρά.

Με τον ίδιο τρόπο θα συνεχίσουμε να εισάγουμε και τις υπόλοιπες πόλεις που θέλουμε στην ουρά.

Θα αποθηκεύσουμε τα k στοιχεία της ουράς σε έναν πίνακα, τον cities, που περιέχει αντικείμενα τύπου City. Ύστερα, καλούμε k φορές την εντολή priorityQueue.getmin(), η οποία επιστρέφει κάθε φορά το μεγαλύτερο στοιχείο της ουράς, το οποίο τοποθετεί στην θέση i του πίνακα cities, ξεκινώντας από το i = k - 1, ώστε με αφετηρία το στοιχείο με δείκτη 0 του πίνακα, οι πόλεις να είναι τοποθετημένες σε αύξουσα σειρά.

Τέλος, τυπώνουμε τα ονόματα και τα IDs των k πόλεων του πίνακα cities, σε αύξουσα σειρά πυκνότητας κρουσμάτων.

Πολυπλοκότητα:

- ➔ Θα πρέπει να διαβάσουμε όλες τις γραμμές του αρχείου εισόδου (N γραμμές δηλαδή), άρα η επαναληπτική διαδικασία για το διάβασμα όλων των γραμμών θα έχει πολυπλοκότητα O(N).
- ➔ Κατά το δεύτερο διάβασμα των γραμμών του αρχείου, χρειάζεται να χρησιμοποιούμε τις μεθόδους insert και getMin, που γνωρίζουμε ότι έχουν και οι δύο πολυπλοκότητα O(logk), καθώς εφαρμόζονται στην ουρά, η οποία περιέχει k στοιχεία.
- ➔ Στο τέλος παίρνουμε τα k στοιχεία από την ουρά, και αυτή η διαδικασία πραγματοποιείται σε χρόνο O(k).

Για την επεξεργασία/δημιουργία της ουράς (πρώτα δύο βελάκια) έχουμε συνολική πολυπλοκότητα O(N*logk). Η O(k) είναι αμελητέα μπροστά στην O(N*logk), άρα την αγνοούμε.

Δηλαδή η συνολική πολυπλοκότητα του αλγορίθμου είναι O(N*logk).

Ο αλγόριθμος του ερωτήματος Α έχει πολυπλοκότητα O(N*logN), αφού στηρίζεται σε αλγόριθμο ταξινόμησης (αλγόριθμος quicksort).

Εάν k<<<<<<N, N*logk << N*logN , άρα συμφέρει ο αλγόριθμος Γ. Θα δεσμεύσει πολύ λίγη μνήμη (ανάλογα με το k), ενώ ο Α θα δεσμεύσει μνήμη ανάλογη με το σύνολο Ν των πόλεων.

ΜΕΡΟΣ Δ

Ανάλυση πολυπλοκότητας αλγορίθμου

Έχουμε υλοποιήσει τις δύο ουρές προτεραιότητας ως binary heaps.

Η εισαγωγή ενός στοιχείου σε ένα binary heap έχει πολυπλοκότητα O(logN), όπου Ν είναι ο αριθμός των στοιχείων στον σωρό. Το logN είναι ο μέγιστος αριθμός swap που μπορούν να γίνουν για να φτάσουμε από την ρίζα του σωρού μέχρι τα φύλλα του (βάθος).

Με την ίδια λογική, η getmin έχει πολυπλοκότητα O(logN), καθώς θα χρειαστούν μέχρι και logN swaps για να αποκατασταθεί η ιδιότητα του σωρού ύστερα από την αφαίρεση ενός στοιχείου (δηλαδή τα στοιχεία να φθίνουν σε προτεραιότητα από πάνω προς τα κάτω στο min heap, και να αυξάνονται στο max heap).

Η balanceHeap καλεί την getmin, η οποία έχει πολυπλοκότητα O(logN). Η balanceHeap έχει επίσης πολυπλοκότητα O(logN), καθώς πραγματοποιεί προσθέσεις και αφαιρέσεις στοιχείων στις ουρές, και είπαμε ότι αυτές οι διαδικασίες έχουν πολυπλοκότητα O(logN).

Καλώντας την calculateMedian, παίρνουμε το μεγαλύτερο στοιχείο του max heap (δηλαδή την ρίζα του σωρού), το μικρότερο στοιχείο του min heap (δηλαδή την ρίζα του άλλου σωρού), και αν χρειάζεται τα συγκρίνουμε. Όλες αυτές οι διαδικασίες πραγματοποιούνται σε σταθερό χρόνο O(1), καθώς έχουμε άμεση πρόσβαση στην ρίζα ενός σωρού. Άρα η πολυπλοκότητα της calculateMedian είναι O(1).

Αφού η updatePQ εισάγει κάθε φορά ένα στοιχείο σε μια ουρά (πρώτο O(logN)) και καλεί την balanceHeap (δεύτερο O(logN)) (συνεπώς και την getmin (τρίτο O(logN))), έχει πολυπλοκότητα O(logN) + O(logN) + O(logN) = 3* O(logN) = O(logN).

Άρα η συνολική πολυπλοκότητα του προγράμματος είναι O(NlogN), αφού έχουμε Ν εισαγωγές, η καθεμία από τις οποίες πραγματοποιείται σε χρόνο O(logN).

Ανάλυση αλγορίθμου

Για την υλοποίηση του προγράμματος κατασκευάσαμε μια μεγιστοστρεφή και μια ελαχιστροστρεφή ουρά προτεραιότητας (max & min heap αντίστοιχα).

Για να κατασκευάσουμε την ελαχιστοστρεφή ουρά (την ονομάσαμε leftHalfPQ), χωρίς να αντιστρέψουμε τα στοιχεία που προκύπτουν από την σύγκριση κάθε δύο πόλεων μεταξύ τους μέσω της compareTo. Έτσι, το πρώτο στοιχείο της ουράς θα είναι το μικρότερο στοιχείο της (δηλαδή η ρίζα), και την μεγαλύτερη προτεραιότητα θα την έχουν οι πόλεις με την μικρότερη πυκνότητα κρουσμάτων.

Ομοίως, κατασκευάζουμε την μεγιστοστρεφή ουρά (την ονομάσαμε rightHalfPQ), αντιστρέψαμε τα στοιχεία που προκύπτουν από την σύγκριση κάθε δύο πόλεων μεταξύ τους μέσω της compareTo. Έτσι, το πρώτο στοιχείο της ουράς θα είναι το μεγαλύτερο στοιχείο της (δηλαδή η ρίζα), και την μεγαλύτερη προτεραιότητα θα την έχουν οι πόλεις με την μεγαλύτερη πυκνότητα κρουσμάτων.

Οι δύο ουρές επιλέχθηκαν με την λογική ότι θα έχουν εκ των πραγμάτων διαδοχικές ρίζες, και ότι

- ➔ Αν έχουμε εισάγει μονό αριθμό στοιχείων στις ουρές και η δεξιά ουρά έχει ένα στοιχείο παραπάνω από την αριστερή, ο median θα είναι η ρίζα της δεξιάς ουράς.
- ➔ Αν έχουμε εισάγει μονό αριθμό στοιχείων στις ουρές και η αριστερή ουρά έχει ένα στοιχείο παραπάνω από την δεξιά, ο median θα είναι η ρίζα της αριστερής ουράς.
- ➔ Αν έχουμε εισάγει άρτιο αριθμό στοιχείων στις ουρές, ο median θα είναι η μικρότερη από τις δύο ρίζες.

Το πρώτο στοιχείο θα το εισάγουμε στην δεξιά (μεγιστοστρεφή) ουρά. Σαν κριτήριο εισαγωγής για τα επόμενα στοιχεία βάλαμε ότι αυτά θα εισάγονται στην δεξιά μεγιστοστρεφή ουρά εάν είναι μικρότερα ή ίσα της ρίζας της, διαφορετικά θα εισάγονται στην ελαχιστοστρεφή ουρά.

Σε κάθε εισαγωγή στοιχείου θα καλείται η συνάρτηση balanceHeap, η οποία θα ελέγχει ότι οι δύο ουρές θα διαφέρουν το πολύ κατά ένα στοιχείο (σε περίπτωση που έχουμε εισάγει συνολικά μονό αριθμό στοιχείων, η μια ουρά θα έχει αναγκαστικά ένα στοιχείο παραπάνω από την άλλη). Εάν δεν υπήρχε αυτός ο έλεγχος, μια από τις ουρές θα μπορούσε να μεγαλώνει απεριόριστα, και δεν θα μπορούσαμε να υπολογίσουμε τον median όπως μας λένε τα παραπάνω βελάκια.

Για να λειτουργεί η balanceHeap, όταν η μια από τις δύο ουρές αποκτήσει δύο παραπάνω στοιχεία από την άλλη, θα αφαιρέσουμε την ρίζα της και θα την προσθέσουμε στην μικρότερη ουρά (έχουμε πει ότι οι ρίζες θα είναι πάντα διαδοχικοί αριθμοί). Έτσι, οι ουρές θα διαφέρουν πάντα το πολύ κατά 1 στοιχείο.

Τέλος, με την χρήση της calculateMedian υλοποιούμε τον τελικό υπολογισμό του median, δηλαδή περνάμε τα παραπάνω βελάκια σε κώδικα.

Παράδειγμα:

1η εισαγωγή (δεν υπάρχαν στοιχεία στις αυρές)
Εισάγαμε το 58 στο max heap.

: 58 ← έγινε αυτομάτως η ρίζα του max Heap

2η εισαγωγή, αριθμός 64

64 : 58

Το 64 μπήκε στο min heap, καθώς 64>58 και λόγω της balance. Είναι πλέον η ρίζα του min heap.

3η εισαγωγή, αριθμός 70 (70>58)

64 70 : 58

4η εισαγωγή, αριθμός 80

μεταφέρθηκε η ρίζα του min heap στο max heap

70 80 : 64 58

↑ νέα ρίζα του max heap

Στην 4η εισαγωγή έπαιξε ρόλο και ότι 80>58 και η balance.

Σαρώθηκε με το CamScanner